

A Deterministic Policy Layer for Reliable In-Car Voice Assistant Behavior in CAR-bench

Sanika Killekar , Siddharth Mohapatra

Northeastern University, Boston, MA

sanikakillekar7@gmail.com, siddharth.m33@gmail.com

Abstract

Large language model agents are increasingly deployed as autonomous, tool-using assistants in interactive settings, yet native LLM tool-calling behavior offers no inherent guarantee of policy compliance, reproducible behavior across repeated interactions, or honest acknowledgment of a system’s actual limitations. CAR-bench [1] makes this gap concrete by evaluating LLM agents as in-car voice assistants across three task types: Base, Hallucination, and Disambiguation. Rather than relying on the underlying LLM’s tool-calling behavior alone, our agent wraps every proposed tool call in a deterministic policy layer that enforces prerequisite checks, missing-parameter and missing-response-field detection, preference-aware auto-injection, post-action side-effect propagation, mutual-exclusion and parallel-call constraints, a two-stage disambiguation system, and a self-consistency-sampled policy-compliance verifier. We describe the design of each component, how they compose within a single LiteLLM [2] completion loop calling Gemini [3], report Pass³ results against the CAR-bench paper’s own baseline for the identical underlying model, and discuss a known gap the design surfaces.

1 Introduction

Large language model agents are increasingly deployed as autonomous, tool-using assistants in interactive settings where they must decide when to act, when to ask, and when to refuse, rather than simply produce a single well-formed response. This shift from single-turn question answering to multi-turn, consequential decision-making exposes a gap current evaluation practice has largely overlooked: whether an agent’s tool-use behavior is not just occasionally correct, but reliably compliant with domain policy and reproducible across repeated interactions with the same request.

In-car voice assistants operate in a setting where a wrong or overconfident action carries real consequences, and where user requests are frequently incomplete, ambiguous, or outside the vehicle’s actual capabilities. CAR-bench [1] evaluates agents on three task categories: Base tasks, which require correct, well-formed tool calls to accomplish a clearly specified goal; Hallucination tasks, which require the agent to recognize when a requested capability, argument, or value

does not exist and to refuse rather than invent a workaround; and Disambiguation tasks, which require asking a clarifying question instead of guessing when a request is genuinely underspecified.

A large language model calling tools natively is not, on its own, reliable at any of these three behaviors: it will often skip a policy-required prerequisite, guess a plausible-sounding but wrong parameter value, or answer a vague request with a specific action rather than a question. Our approach treats the LLM as a proposal mechanism and adds a separate, deterministic policy layer that inspects every proposed tool call (or lack thereof) before it is either forwarded to the CAR-bench evaluator or replaced with a corrected action, a clarifying question, or a refusal.

CAR-bench scores an agent on both single-trial correctness (Pass@1) and Pass³, which requires identical, correct behavior across three independent trials of the same task [1]. Formally, each task t is run $k = 3$ times, yielding binary outcomes $r_t^{(1)}, \dots, r_t^{(k)}$, and

$$\text{Pass}_t^k = \mathbb{1}\left[\sum_{i=1}^k r_t^{(i)} = k\right], \quad \text{Pass}@k_t = \mathbb{1}\left[\sum_{i=1}^k r_t^{(i)} \geq 1\right], \quad (1)$$

each averaged over all tasks in a split to give the reported Pass³ and Pass@3 scores. A check that is right on average but samples different behavior from trial to trial will fail Pass³ even when every individual trial would have scored well in isolation. This consideration shaped several design decisions described below, particularly the preference for deterministic lookups and temperature-0.0 fallbacks over resampling the LLM.

2 Related Work

Our design sits at the intersection of two lines of practice around tool-using LLM agents. The first is native function/tool calling, where an LLM directly emits structured calls that a host application executes [8]; this is the default interaction pattern CAR-bench and most modern agent frameworks build on via LiteLLM [2], but it leaves policy compliance and consistency entirely up to the model’s own judgment on each call. The second is the broader idea of a guardrail or policy layer external to the model, which inspects or rewrites model output before it takes effect [7]. Our agent combines both: native tool calling for proposing actions, and an

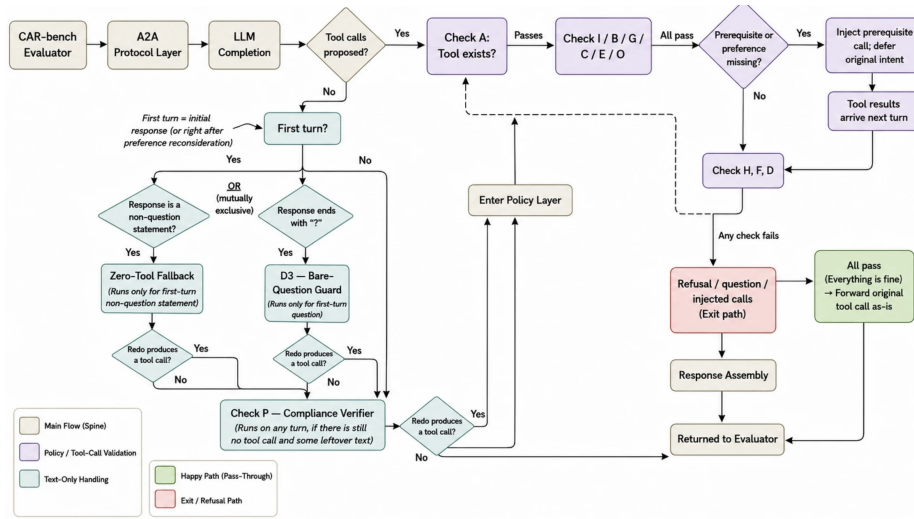


Figure 1: Control flow of the agent, from the A2A protocol layer through the tool-call and text-only response paths to response assembly; check labels correspond to the checks described in Section 4.

external deterministic layer for everything the benchmark’s policy requires but a single LLM call cannot reliably guarantee.

3 System Architecture

The overall architecture separates language generation from policy enforcement, so that reliability guarantees do not depend on the underlying LLM’s own judgment call by call. On the first turn of a task, an A2A protocol layer parses the evaluator’s system prompt, the user’s opening message, and the list of tools available for that task (tool availability varies per task, since Hallucination tasks deliberately remove tools or fields the agent might otherwise expect) into a single LiteLLM [2] completion call to the underlying model, currently Gemini 2.5 Flash [3], with a bounded number of retries and a fixed timeout so a single transient provider error cannot stall a task. On later turns it instead receives a further user message or the structured results of prior tool calls. A deterministic policy layer, described in Section 4, then inspects the model’s proposed output before it is either forwarded to the evaluator or replaced with a corrected action, a clarifying question, or a refusal.

When the model proposes one or more tool calls, they are passed through the methodology described in Section 4 before being forwarded to the evaluator. When the model instead returns plain text, two narrowly-scoped checks intervene: one detects a non-answer, non-question statement on the first turn or immediately after new preference information becomes available, and forces a redo rather than letting the agent silently claim an action without a tool call; the other, described in Section 4.6, detects a bare clarifying question asked before the agent has attempted to look up any relevant stored information.

4 Methodology

The deterministic policy layer decomposes policy enforcement into an ordered sequence of specialized checks, each targeting a distinct class of failure observed in CAR-bench. Each check either passes the proposed call(s) through unchanged, replaces them with a different call (e.g. a prerequisite), replaces them with text (a refusal or a clarifying question), or defers the original call and reintroduces it once a prerequisite is satisfied. The following subsections describe these checks in execution order.

4.1 Tool Existence and Confirmation

Every proposed tool call is first checked against the actual list of tools available for that task. If the model names a tool that has been removed, the call is rejected and replaced with a plain refusal. Separately, a small set of consequential tools require an explicit affirmative reply from the user in the immediately preceding turn; absent that, the agent asks for confirmation instead of acting.

4.2 Prerequisite Injection

Beyond tool validation, many vehicle actions are only safe or policy-compliant once some other piece of state has been confirmed first. To enforce this, the system maintains an explicit table of these dependencies and injects the required lookup automatically whenever a tool that needs one is called.

A table maps specific tools to the prerequisite information the domain policy requires before that tool may be called, for example checking current weather before opening the sunroof. Prerequisites marked automatic are injected directly as additional tool calls in the same turn, batched together so a tool needing several independent prerequisite lookups does not cost several sequential round-trips; prerequisites that require the user’s input instead produce a natural-language question. If the prerequisite tool is itself unavailable, the agent refuses rather than silently constructing a call to a tool that does not exist.

For tools outside this fixed table, a lightweight judge call checks, once per tool per conversation, whether the system prompt’s own policy text implies a prerequisite the fixed table does not cover, and produces a natural clarification if so.

4.3 Missing Information Detection

A proposed call can still be missing required arguments, or silently assume a value that was never actually confirmed. Two related checks catch these. The first flags when a tool’s required parameters are absent from the proposed call and asks the user for exactly the missing values. The second tracks, per conversation, which fields of a prior GET-style tool result actually came back populated versus unknown, and blocks a dependent action (for example, adjusting the defrost setting) if a field it depends on was never confirmed, rather than letting the agent assume a plausible default.

4.4 Preference-Aware Auto-Injection

If required information remains unavailable in the request itself, it may already exist as a stored preference in the user’s vehicle profile. At the same time, an explicit instruction from the user in the current request should always outrank a stored default.

For a fixed set of preference-sensitive tools, the agent auto-injects a lookup of the user’s stored preferences before acting, but only when the proposed call is otherwise incomplete. When the lookup does fire, its result is free-text rather than structured data, so the original call is not blindly replayed once the lookup returns; the model is instead given a fresh turn to reconsider its answer in light of what the preference lookup returned, with a narrow check to catch a subsequent no-op response that neither acts nor asks a follow-up question.

4.5 Mutual Exclusion and Parallel-Call Constraints

Two structural constraints address otherwise-valid calls that conflict with each other or with the vehicle’s own physical state. High beams may not be enabled while fog lights are already on, checked directly against the vehicle’s own last reported light status. Separately, the five navigation-editing tools (adding, replacing, or deleting a destination or way-point) must never be invoked in parallel within a single turn; if the model proposes more than one, only the first is kept and the rest are dropped from that turn.

4.6 Disambiguation

Once a call is structurally complete, a separate question remains: whether the request itself was specific enough to act on at all. Not every underspecified request should be answered with a guess, and CAR-bench specifically penalizes an agent that acts on a genuinely ambiguous instruction instead of asking. The system prompt also includes a fixed set of few-shot examples demonstrating correct clarifying-question behavior across common ambiguity categories (temperature, visibility, comfort, air quality, noise, lighting), each contrasted with an incorrect response; this biases the underlying model itself toward asking narrow, binary questions before the checks below ever run. Disambiguation is handled

in two stages, moving from a cheap deterministic check to a more expensive model-driven one only when necessary.

A short-message heuristic first decides whether the request looks genuinely vague, based on message length and whether it names a specific car system by keyword; requests that are answering a question the agent itself just asked are excluded from this check. If the request looks vague, a pre-computed cache of common vague phrases is checked first, using an ordered strategy that starts with anything already asked earlier in the same conversation, then an exact keyword match, then a word-level match, then a fuzzy substring match. A cache hit returns a vetted, deterministic question at zero additional inference cost.

If the cache does not resolve the ambiguity, the request falls through to an EVPI-style [4] procedure. One call generates m candidate interpretations of the request and, for each of several candidate clarifying questions, a predicted yes/no answer under every interpretation. Each candidate q is then scored deterministically in Python by how evenly it splits those predicted answers,

$$\text{split}(q) = \left| \left| \{i : a_{q,i} = \text{yes}\} \right| - \left| \{i : a_{q,i} = \text{no}\} \right| \right|, \quad (2)$$

and the candidate minimizing $\text{split}(q)$ is selected as the one with the highest expected information gain. The winning question is phrased naturally in a second call, and this fall-back runs at temperature 0.0 rather than a higher sampling temperature, since Pass³ requires the same uncached request to produce the same clarification on every trial.

Finally, a structural gap this design specifically addresses is that the checks above are only reachable when the model proposes a tool call in the first place; if the model instead responds with a bare clarifying question in plain text on the very first turn, none of them run. A dedicated check detects exactly this pattern and forces one redo asking the model to first attempt an information-gathering tool call before trusting that a question is actually necessary; if that redo does produce a tool call, it is routed through the full policy layer like any other proposed call.

4.7 Post-Action Side Effects

Some tools implicitly require additional state changes to remain policy-compliant; this stage automatically augments the relevant tool calls with them. Three specific state-changing actions carry such mandated side effects: enabling air conditioning must also close any window open more than a small threshold and ensure the fan is not off; enabling window defrost must also raise fan speed, switch airflow toward the windshield, and turn on air conditioning if it is not already on; enabling fog lights must also ensure low beams are on and high beams are off. Because the prerequisite checks in Section 4.2 already guarantee the underlying state has been fetched before these trigger tools can be called, the side-effect calculation can run purely from already-cached tool results rather than requiring another round of lookups, and the additional calls are appended to the same batch of tool calls the evaluator executes together.

4.8 Policy-Compliance Verifier

Finally, even after every preceding check succeeds, the agent’s final response can still misrepresent what actually

happened, either by falsely claiming a state-changing action was completed with no tool call to back it up, or by failing to satisfy a semantic policy that does not reduce to a deterministic table. A final verification stage therefore checks both of these at once via a structured-output judge call, for a small set of semantic wiki policies (for example, informing the user about toll roads on a presented route). Each semantic rule is checked at most once per conversation and only when its trigger tool has actually been used. If a violation is flagged, two independent redo attempts are sampled and the first one that actually produces a tool call is preferred over one that merely rephrases the text, since a flagged completion-claim is more reliably fixed by an action than by different wording.

Together, these checks form the deterministic policy layer executed before every tool call or text response is returned to the evaluator.

5 Validation Results

We compare our full system’s Pass³ scores against the CAR-bench paper’s own reported baseline for the identical underlying model, Gemini 2.5 Flash (non-thinking), run with native tool calling only and no custom scaffolding [1]. All numbers below are on the public task split; hidden-test-set scores are withheld per the competition’s reporting rules.

Task Type	N	Baseline	Ours	Δ
Base	50	48%	46%	−2 pts
Hallucination	150	22%	42%	+20 pts
Disambiguation	25	22%	36%	+14 pts

Table 1: Pass³ comparison against the CAR-bench paper’s Gemini-2.5-flash baseline, identical underlying model. N is the number of tasks in the public split for that category.

On Hallucination, the tool-existence and prerequisite-removal handling in Sections 4.1–4.2, whose explicit purpose is to block calls to tools or prerequisites the evaluator has removed, nearly doubles the baseline Pass³ (22%→42%); the same run’s Pass@1 and Pass@3 (64% and 74%, respectively, versus a baseline of 37% and 52%) confirm this is a broad improvement. On Disambiguation, the cache-first and EVPI-based clarification strategy described in Section 4.6 similarly improves Pass³ substantially over baseline (22%→36%), indicating that resolving ambiguity deterministically before falling back to the LLM helps the agent give the same clarifying question consistently across trials. On Base, Pass@1 and Pass@3 improve substantially over baseline as well (80% and 92% versus 53% and 62%), but Pass³ itself is essentially flat, slightly below baseline. This is a genuine, unresolved gap rather than a favorable result: the agent frequently succeeds on at least one of the three trials but does not reliably reproduce the identical successful behavior on all three, indicating that the policy layer’s checks for Base tasks reduce average-case failure without yet closing the specific trial-to-trial consistency gap Pass³ is designed to expose.

6 Efficiency, Limitations, and Discussion

All judge-style calls (the generic prerequisite fallback, the policy-compliance verifier, and the EVPI candidate generator) use structured output schemas rather than a plain instruction to reply with JSON, avoiding malformed or fence-wrapped output. The disambiguation cache in Section 4.6 specifically reduces cost and latency by resolving the majority of vague requests with zero additional model calls rather than always invoking the two-call EVPI fallback. Concretely, across a completed 150-task Hallucination run (3 trials each, 300 task-runs measured), the agent averaged \$0.01 in LLM cost and 16.5 seconds of total LLM-induced latency per task-run, using roughly 44,500 tokens (41,438 input, 3,070 output) and 4.4 A2A turns on average. Checks whose correctness matters most for Pass³ consistency sample multiple independent completions [5] and prefer whichever result is non-empty or actually contains a tool call, rather than trusting a single sample. Finally, a short, capped, deduplicated log of lessons learned from violations the compliance verifier catches in Section 4.8 is carried across tasks within the same running process and surfaced back into the system prompt for later tasks, giving the agent a lightweight Reflexion-style [6] memory of its own past policy mistakes without any weight updates or code changes between tasks.

The Base results in Section 5 point to a first limitation directly: substantial Pass@1/Pass@3 gains without a matching Pass³ gain indicates that at least one further source of trial-to-trial variance in Base tasks is not yet covered by a deterministic check, unlike the disambiguation cache or the temperature-0.0 EVPI fallback. A second, related gap is that the agent can propose a fully-specified but substantively wrong tool call for an ambiguous request, for example directly guessing a specific value instead of first checking a relevant stored preference. The preference auto-injection in Section 4.4 only triggers when a call is incomplete; a call that is complete but incorrect currently passes through unchecked, since completeness rather than correctness is what that specific check inspects. Extending disambiguation coverage to this case, and identifying the remaining source(s) of Base inconsistency, are the two most direct next steps for improving Pass³ further.

7 Conclusion

Treating the LLM strictly as a proposal mechanism, and moving every deterministic or policy-derivable decision into a separate layer, lets the agent recover from common LLM tool-use failure modes: skipped prerequisites, incomplete calls, missing-field blindness, unwarranted completion claims, and inconsistent handling of ambiguous requests. Measured against the CAR-bench paper’s own baseline, this nearly doubles Hallucination Pass³ and meaningfully improves Disambiguation Pass³, while leaving Base Pass³ roughly unchanged despite large Pass@1/Pass@3 gains – a result we report directly, since it identifies where the design’s reliability guarantees do not yet reach. Reliability here means reproducibility across repeated trials, not only correctness on any single one, which is why the design leans on deterministic cache-first disambiguation, temperature-0.0 fall-

backs, and self-consistency sampling wherever trial-to-trial variance was the specific failure mode.

Acknowledgements

We thank the CAR-bench competition organizers for designing this evaluation framework. This work was completed as part of the IJCAI-ECAI 2026 CAR-bench competition, Track 1, at Northeastern University.

References

- [1] Johannes Kirmayr, Lukas Stappen, and Elisabeth Andre. CAR-bench: Evaluating the Consistency and Limit-Awareness of LLM Agents under Real-World Uncertainty. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 40599–40618, San Diego, California, United States, July 2026. Association for Computational Linguistics. <https://aclanthology.org/2026.acl-long.1886/>
- [2] BerriAI. *LiteLLM: Call all LLM APIs using the OpenAI format*. <https://github.com/BerriAI/litellm>
- [3] Google. *Gemini 2.5 Flash API Documentation*. <https://ai.google.dev/gemini-api/docs>
- [4] Jan B. Oostenbrink, Maiwenn J. Al, Mark Oppe, and Maureen P. M. H. Rutten-van Mölken. Expected value of perfect information: an empirical example of reducing decision uncertainty by conducting additional research. *Value in Health*, 11(7):1070–1080, 2008. <https://doi.org/10.1111/j.1524-4733.2008.00389.x>
- [5] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *Proceedings of the Eleventh International Conference on Learning Representations (ICLR)*, 2023. <https://arxiv.org/abs/2203.11171>
- [6] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. <https://arxiv.org/abs/2303.11366>
- [7] Traian Rebedea, Razvan Dinu, Makesh Sreedhar, Christopher Parisien, and Jonathan Cohen. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 431–445, 2023. <https://arxiv.org/abs/2310.10501>
- [8] Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, Jun Xu, and Ji-Rong Wen. Tool Learning with Large Language Models: A Survey. *Frontiers of Computer Science*, 19(8):198343, 2025. <https://arxiv.org/abs/2405.17935>